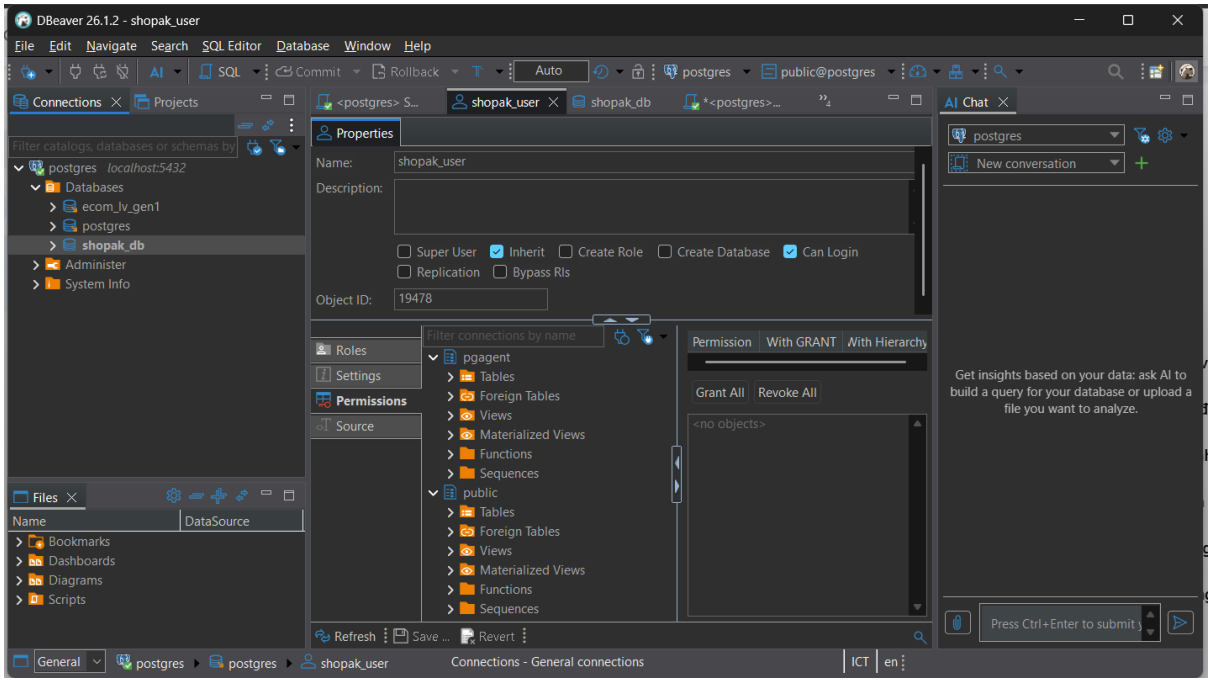
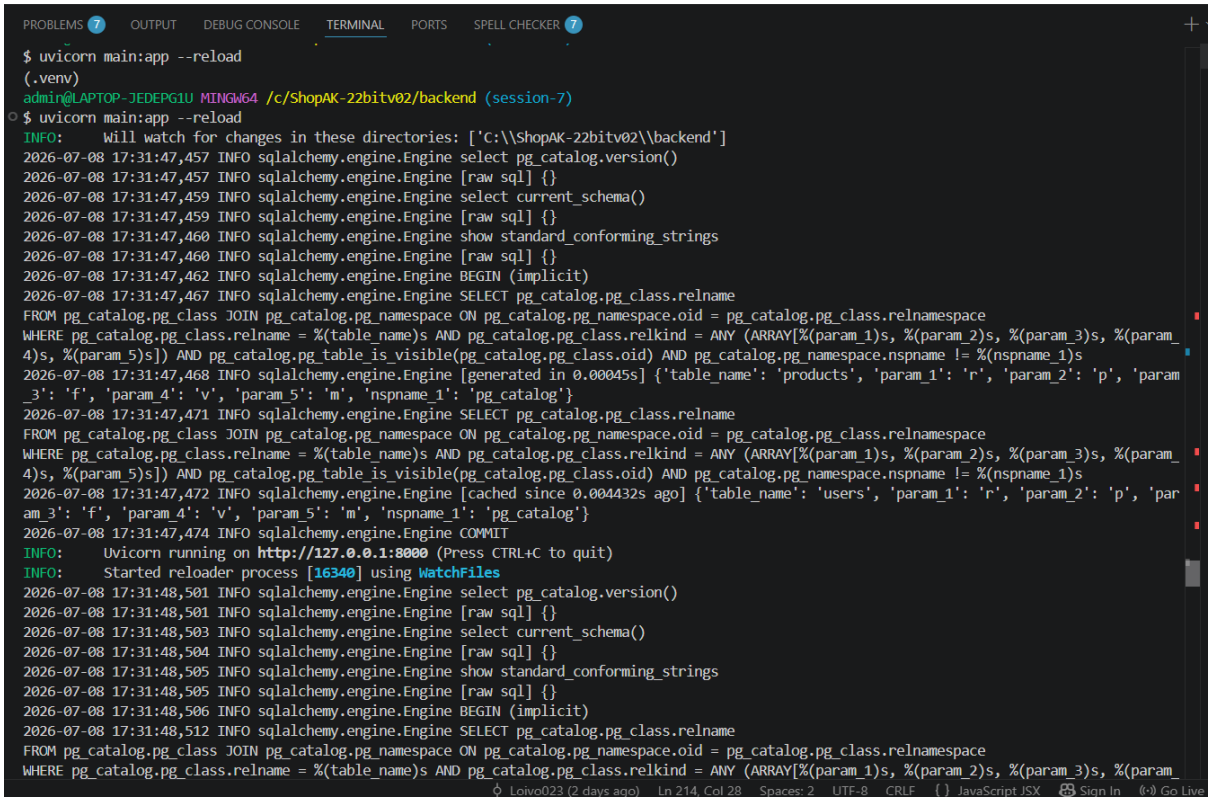
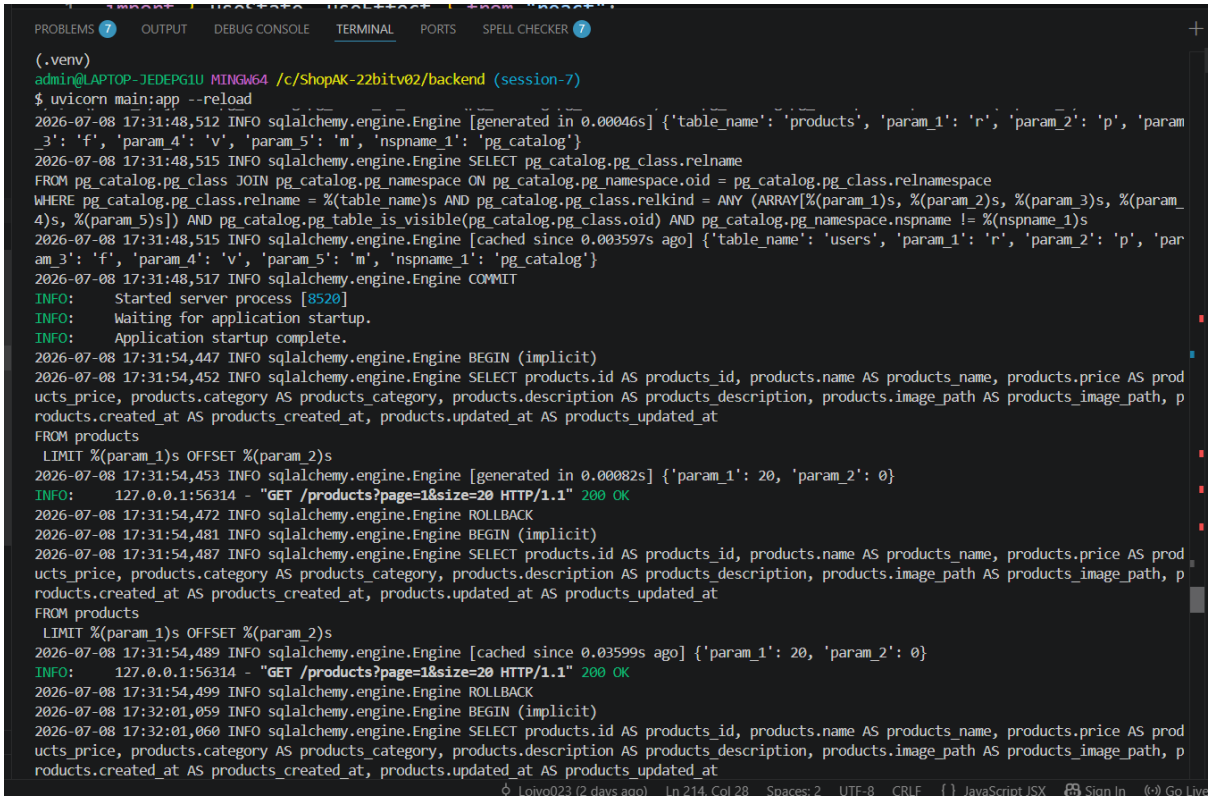
Set up PostgreSQL và kết nối FastAPI (confirm logs show DB connection)

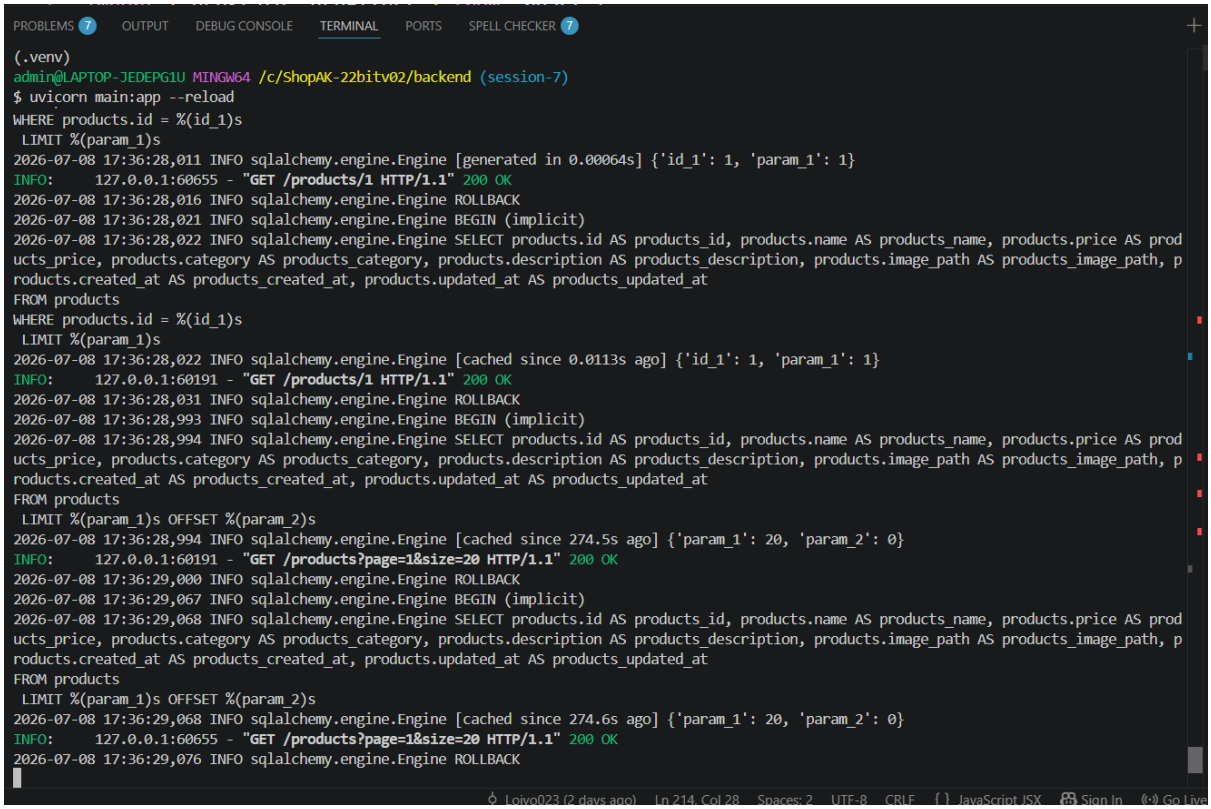
Đây là database chính của web



Terminal chạy uvicorn main:app reload database.py sẽ có echo=True, terminal sẽ hiện log SQL như CREATE TABLE, BEGIN, COMMIT,... Backend đã kết nối thành công tới PostgreSQL

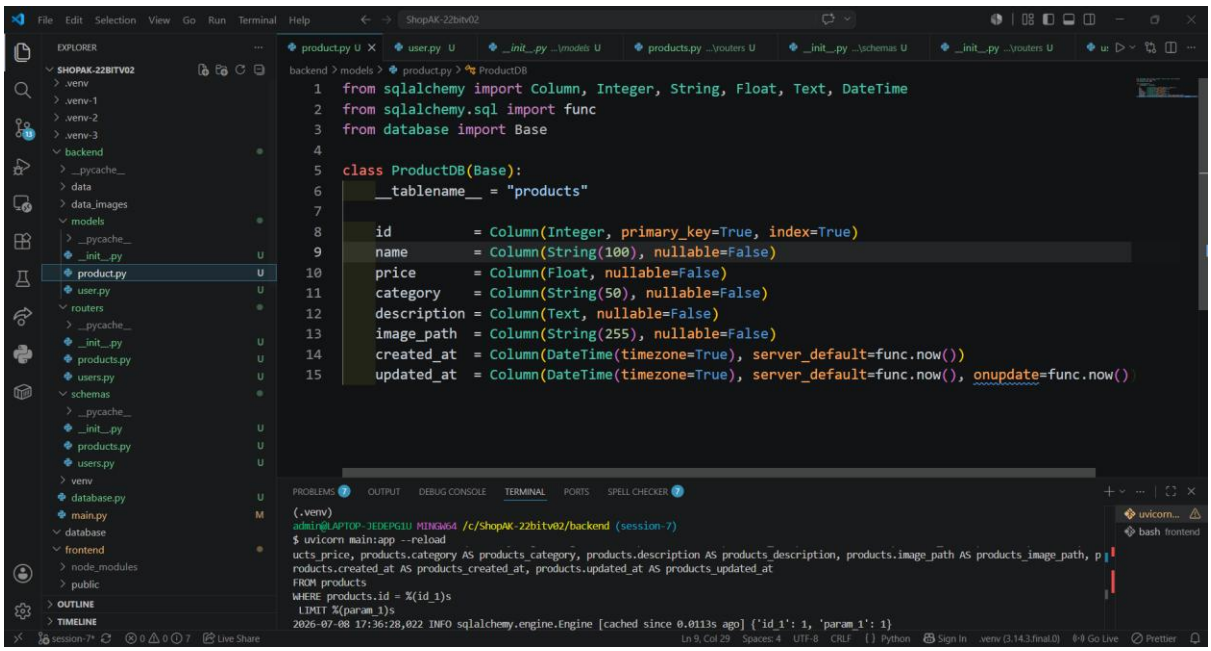

PROBLEMS 7
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
SPELL CHECKER 7
$ uvicorn main:app --reload
(.venv)
admin@LAPTOP-JEDEPG1U MINGW64 /c/ShopAK-22bitv02/backend (session-7)
$ uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['C:\\ShopAK-22bitv02\\backend']
2026-07-08 17:31:47,457 INFO sqlalchemy.engine.Engine select pg_catalog.version()
2026-07-08 17:31:47,457 INFO sqlalchemy.engine.Engine [raw sql] {}
2026-07-08 17:31:47,459 INFO sqlalchemy.engine.Engine select current_schema()
2026-07-08 17:31:47,459 INFO sqlalchemy.engine.Engine [raw sql] {}
2026-07-08 17:31:47,460 INFO sqlalchemy.engine.Engine show standard_conforming_strings
2026-07-08 17:31:47,460 INFO sqlalchemy.engine.Engine [raw sql] {}
2026-07-08 17:31:47,462 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-07-08 17:31:47,467 INFO sqlalchemy.engine.Engine SELECT pg_catalog.pg_class.relname
FROM pg_catalog.pg_class JOIN pg_catalog.pg_namespace ON pg_catalog.pg_namespace.oid = pg_catalog.pg_class.relnamespace
WHERE pg_catalog.pg_class.relname = %(table_name)s AND pg_catalog.pg_class.relkind = ANY (ARRAY[%(param_1)s, %(param_2)s, %(param_3)s, %(param_
4)s, %(param_5)s]) AND pg_catalog.pg_table_is_visible(pg_catalog.pg_class.oid) AND pg_catalog.pg_namespace.nspname != %(nspname_1)s
2026-07-08 17:31:47,468 INFO sqlalchemy.engine.Engine [generated in 0.00045s] {'table_name': 'products', 'param_1': 'r', 'param_2': 'p', 'param
_3': 'f', 'param_4': 'v', 'param_5': 'm', 'nspname_1': 'pg_catalog'}
2026-07-08 17:31:47,471 INFO sqlalchemy.engine.Engine SELECT pg_catalog.pg_class.relname
FROM pg_catalog.pg_class JOIN pg_catalog.pg_namespace ON pg_catalog.pg_namespace.oid = pg_catalog.pg_class.relnamespace
WHERE pg_catalog.pg_class.relname = %(table_name)s AND pg_catalog.pg_class.relkind = ANY (ARRAY[%(param_1)s, %(param_2)s, %(param_3)s, %(param_
4)s, %(param_5)s]) AND pg_catalog.pg_table_is_visible(pg_catalog.pg_class.oid) AND pg_catalog.pg_namespace.nspname != %(nspname_1)s
2026-07-08 17:31:47,472 INFO sqlalchemy.engine.Engine [cached since 0.004432s ago] {'table_name': 'users', 'param_1': 'r', 'param_2': 'p', 'par
am_3': 'f', 'param_4': 'v', 'param_5': 'm', 'nspname_1': 'pg_catalog'}
2026-07-08 17:31:47,474 INFO sqlalchemy.engine.Engine COMMIT
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [16340] using WatchFiles
2026-07-08 17:31:48,501 INFO sqlalchemy.engine.Engine select pg_catalog.version()
2026-07-08 17:31:48,501 INFO sqlalchemy.engine.Engine [raw sql] {}
2026-07-08 17:31:48,503 INFO sqlalchemy.engine.Engine select current_schema()
2026-07-08 17:31:48,504 INFO sqlalchemy.engine.Engine [raw sql] {}
2026-07-08 17:31:48,505 INFO sqlalchemy.engine.Engine show standard_conforming_strings
2026-07-08 17:31:48,505 INFO sqlalchemy.engine.Engine [raw sql] {}
2026-07-08 17:31:48,506 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-07-08 17:31:48,512 INFO sqlalchemy.engine.Engine SELECT pg_catalog.pg_class.relname
FROM pg_catalog.pg_class JOIN pg_catalog.pg_namespace ON pg_catalog.pg_namespace.oid = pg_catalog.pg_class.relnamespace
WHERE pg_catalog.pg_class.relname = %(table_name)s AND pg_catalog.pg_class.relkind = ANY (ARRAY[%(param_1)s, %(param_2)s, %(param_3)s, %(param_
Loiyo023 (2 days ago) Ln 214, Col 28 Spaces: 2 UTF-8 CRLF { } JavaScript JSX Sign In Go Live


PROBLEMS 7
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
SPELL CHECKER 7
(.venv)
admin@LAPTOP-JEDEPG1U MINGW64 /c/ShopAK-22bitv02/backend (session-7)
$ uvicorn main:app --reload
2026-07-08 17:31:48,512 INFO sqlalchemy.engine.Engine [generated in 0.00046s] {'table_name': 'products', 'param_1': 'r', 'param_2': 'p', 'param
_3': 'f', 'param_4': 'v', 'param_5': 'm', 'nspname_1': 'pg_catalog'}
2026-07-08 17:31:48,515 INFO sqlalchemy.engine.Engine SELECT pg_catalog.pg_class.relname
FROM pg_catalog.pg_class JOIN pg_catalog.pg_namespace ON pg_catalog.pg_namespace.oid = pg_catalog.pg_class.relnamespace
WHERE pg_catalog.pg_class.relname = %(table_name)s AND pg_catalog.pg_class.relkind = ANY (ARRAY[%(param_1)s, %(param_2)s, %(param_3)s, %(param_
4)s, %(param_5)s]) AND pg_catalog.pg_table_is_visible(pg_catalog.pg_class.oid) AND pg_catalog.pg_namespace.nspname != %(nspname_1)s
2026-07-08 17:31:48,515 INFO sqlalchemy.engine.Engine [cached since 0.003597s ago] {'table_name': 'users', 'param_1': 'r', 'param_2': 'p', 'par
am_3': 'f', 'param_4': 'v', 'param_5': 'm', 'nspname_1': 'pg_catalog'}
2026-07-08 17:31:48,517 INFO sqlalchemy.engine.Engine COMMIT
INFO: Started server process [8520]
INFO: Waiting for application startup.
INFO: Application startup complete.
2026-07-08 17:31:54,447 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-07-08 17:31:54,452 INFO sqlalchemy.engine.Engine SELECT products.id AS products_id, products.name AS products_name, products.price AS prod
ucts_price, products.category AS products_category, products.description AS products_description, products.image_path AS products_image_path, p
roducts.created_at AS products_created_at, products.updated_at AS products_updated_at
FROM products
LIMIT %(param_1)s OFFSET %(param_2)s
2026-07-08 17:31:54,453 INFO sqlalchemy.engine.Engine [generated in 0.00082s] {'param_1': 20, 'param_2': 0}
INFO: 127.0.0.1:56314 - "GET /products?page=1&size=20 HTTP/1.1" 200 OK
2026-07-08 17:31:54,472 INFO sqlalchemy.engine.Engine ROLLBACK
2026-07-08 17:31:54,481 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-07-08 17:31:54,487 INFO sqlalchemy.engine.Engine SELECT products.id AS products_id, products.name AS products_name, products.price AS prod
ucts_price, products.category AS products_category, products.description AS products_description, products.image_path AS products_image_path, p
roducts.created_at AS products_created_at, products.updated_at AS products_updated_at
FROM products
LIMIT %(param_1)s OFFSET %(param_2)s
2026-07-08 17:31:54,489 INFO sqlalchemy.engine.Engine [cached since 0.03599s ago] {'param_1': 20, 'param_2': 0}
INFO: 127.0.0.1:56314 - "GET /products?page=1&size=20 HTTP/1.1" 200 OK
2026-07-08 17:31:54,499 INFO sqlalchemy.engine.Engine ROLLBACK
2026-07-08 17:32:01,059 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-07-08 17:32:01,060 INFO sqlalchemy.engine.Engine SELECT products.id AS products_id, products.name AS products_name, products.price AS prod
ucts_price, products.category AS products_category, products.description AS products_description, products.image_path AS products_image_path, p
roducts.created_at AS products_created_at, products.updated_at AS products_updated_at
Loiyo023 (2 days ago) Ln 214, Col 28 Spaces: 2 UTF-8 CRLF { } JavaScript JSX Sign In Go Live

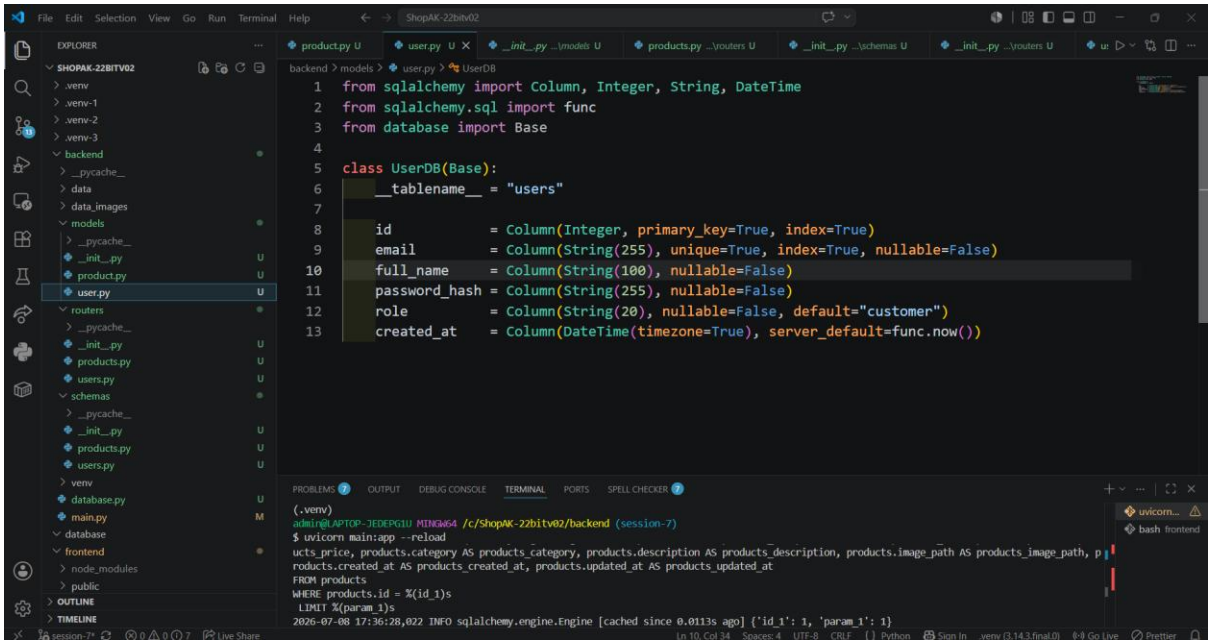


Định nghĩa ProductDB và UserDB models + tạo tables

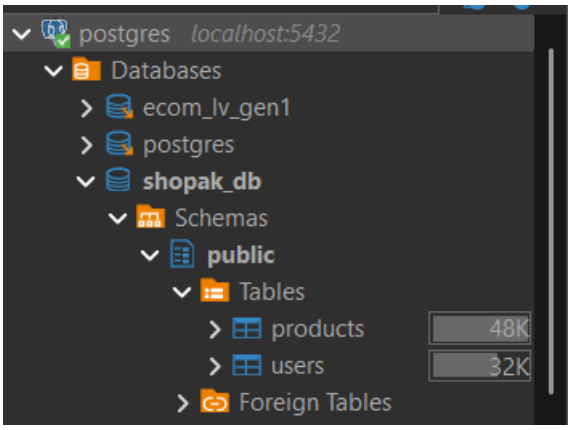
Dưới đây là đoạn code của Class ProductDB



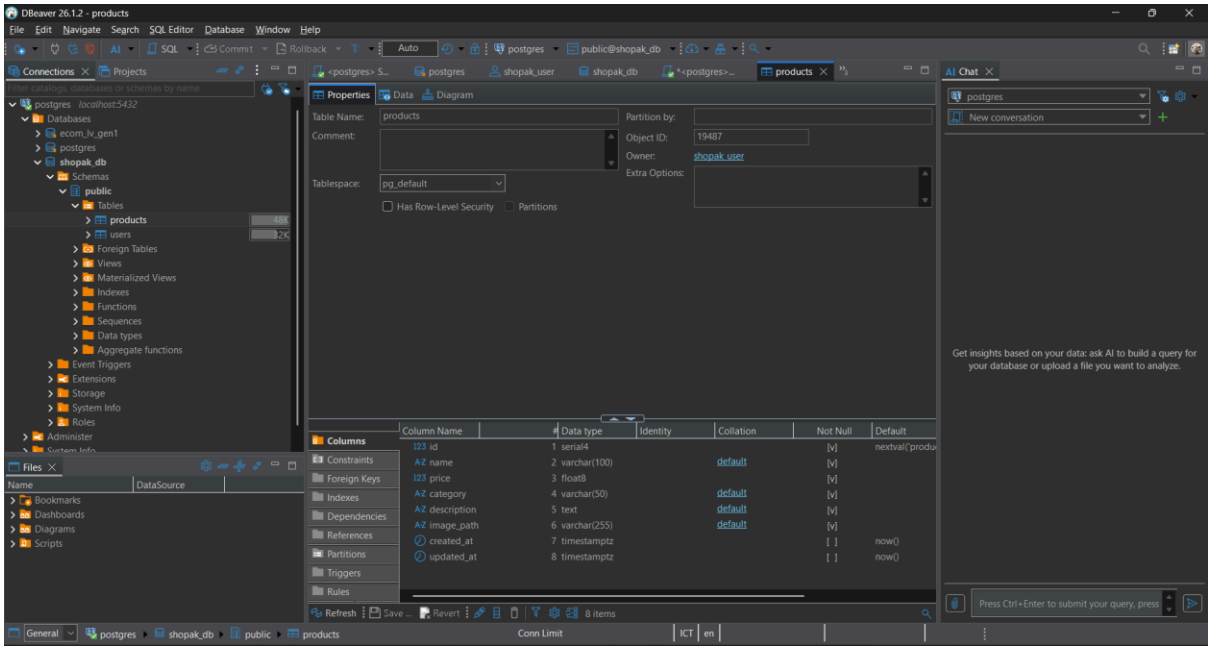
Dưới đây là đoạn code của Class userDB



Tiếp theo là phần tạo bảng dưới đây ta có thể thấy 2 bảng product và users của sql



Dưới đây ta có thể cấu trúc cột của bảng product (id, name, price, category, description, image_path, created_at, updated_at)



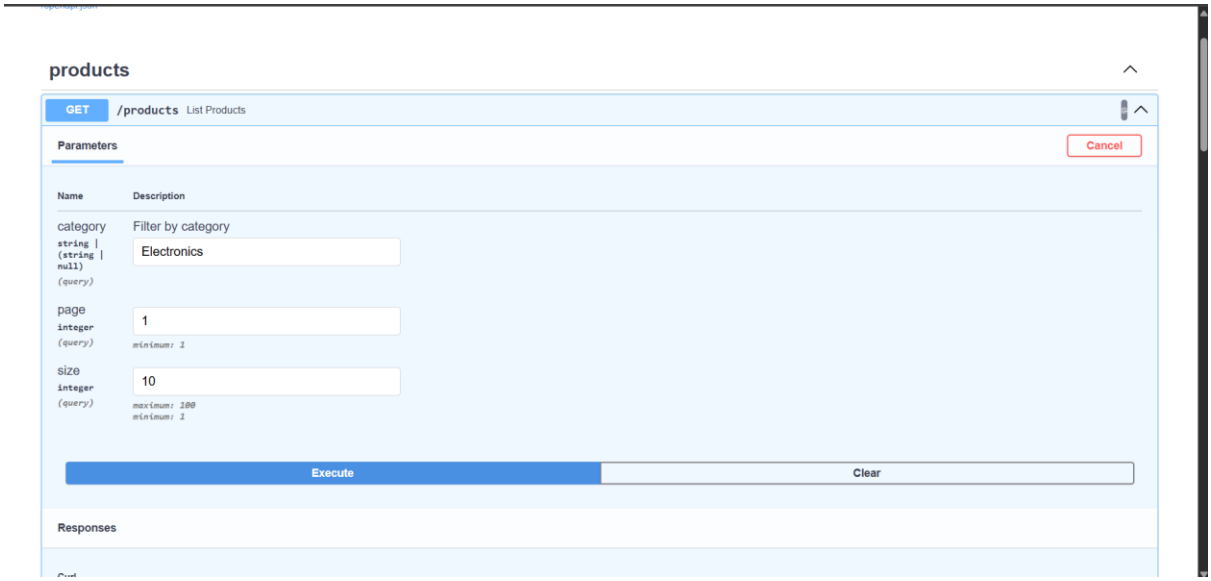
Implement Product CRUD dùng DB sessions thay vì JSON files

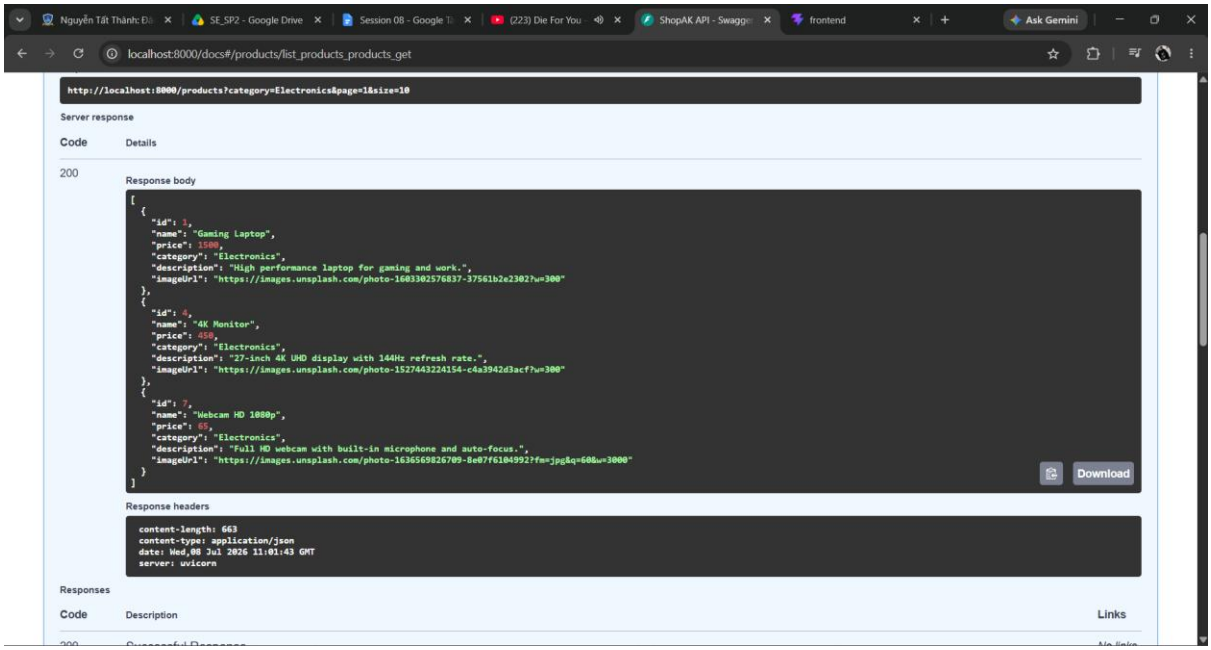
```python
backend > routers > products.py > list_products
    )


@router.post("", response_model=ProductRead, status_code=201)
def create_product(payload: ProductCreate, db: Session = Depends(get_db)):
    new_product = ProductDB(
        name=payload.name, price=payload.price,
        category=payload.category, description=payload.description,
        image_path=payload.imageUrl,
    )
    db.add(new_product)
    db.commit()
    db.refresh(new_product)
    return ProductRead(
        id=new_product.id, name=new_product.name, price=new_product.price,
        category=new_product.category, description=new_product.description,
        imageUrl=new_product.image_path,
    )
```

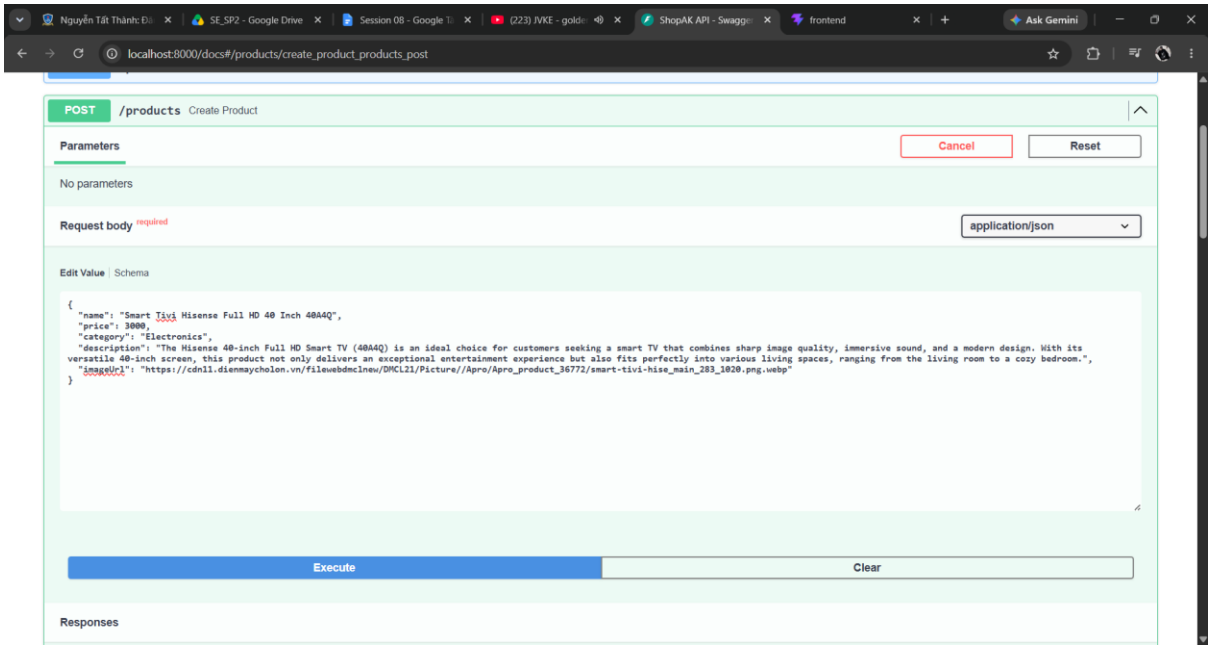
Test tất cả endpoints qua Swagger, confirm data persists

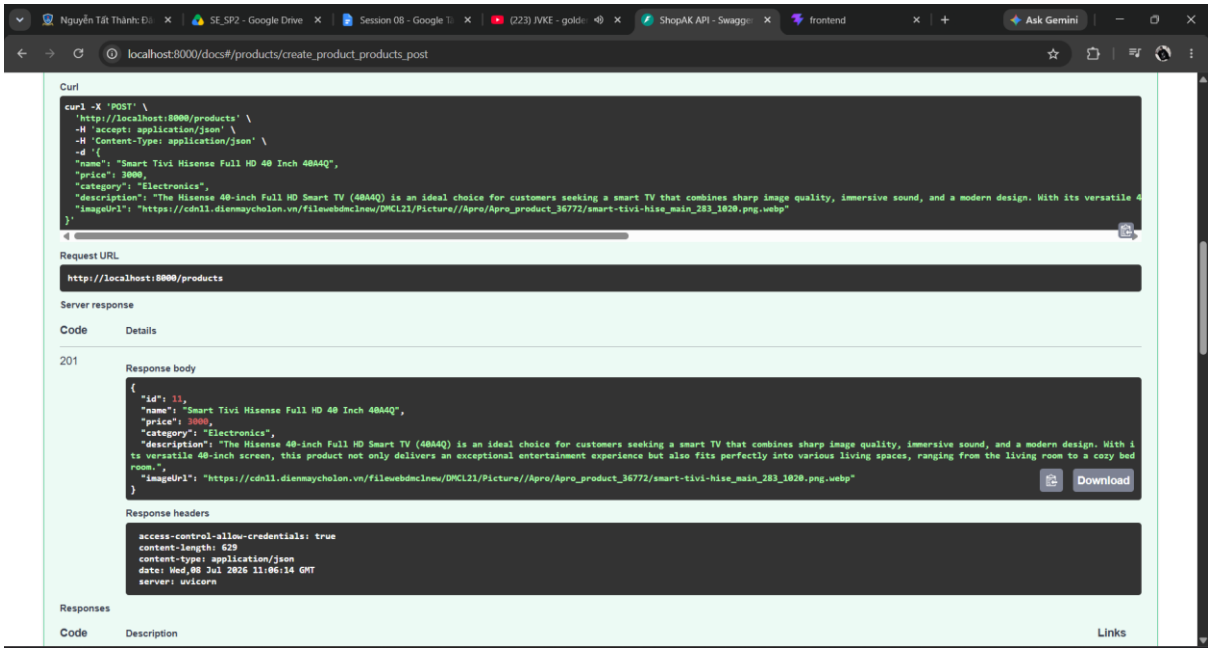
Đầu tiên chúng ta sẽ test GET/Products để tìm ra sản phẩm Electronics và trả kết quả sản phẩm về 200



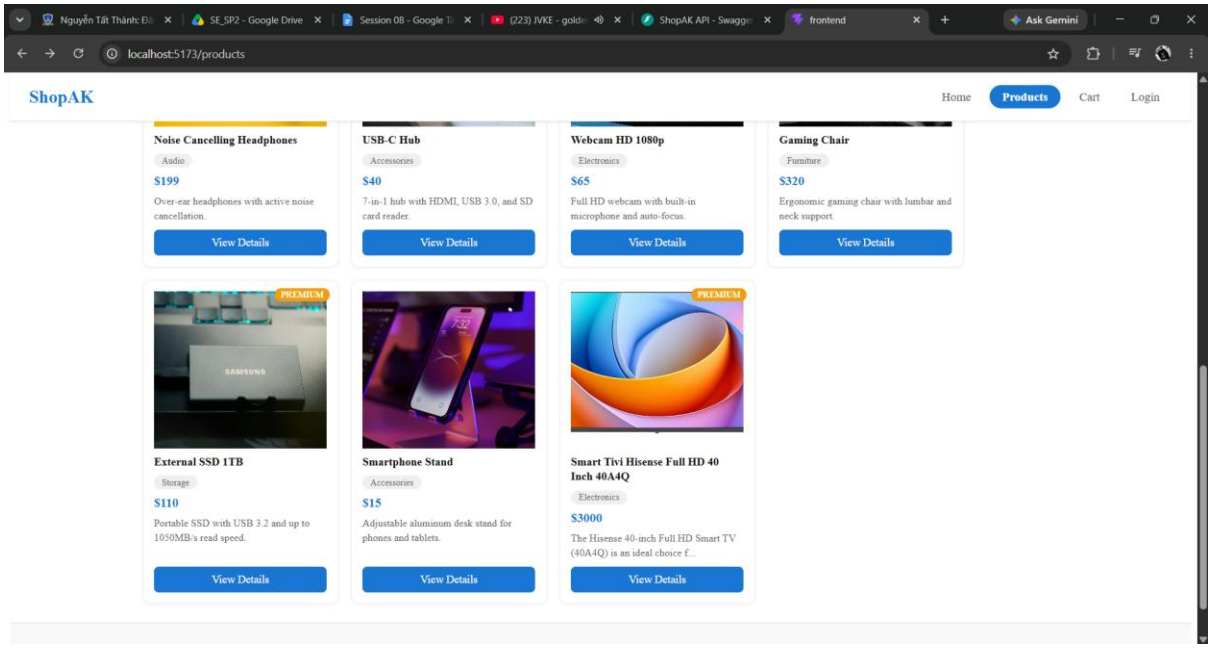


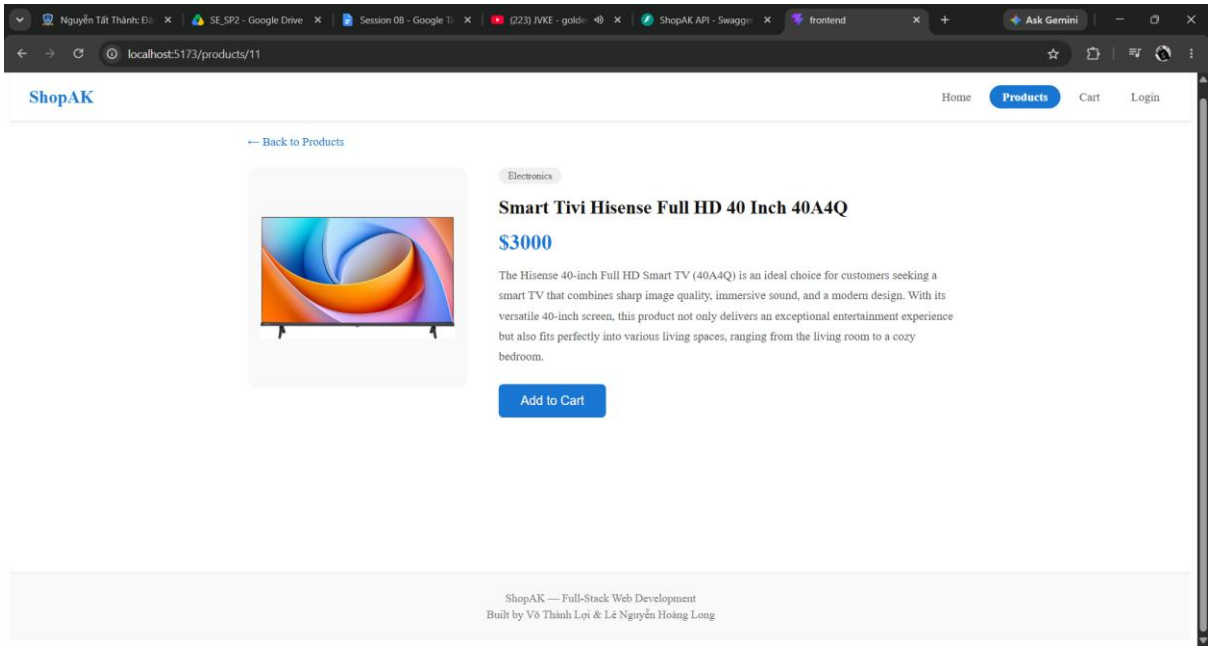
Tiếp theo chúng ta sẽ test về POST/Products để tạo ra 1 sản phẩm mới và trả về kết quả 201



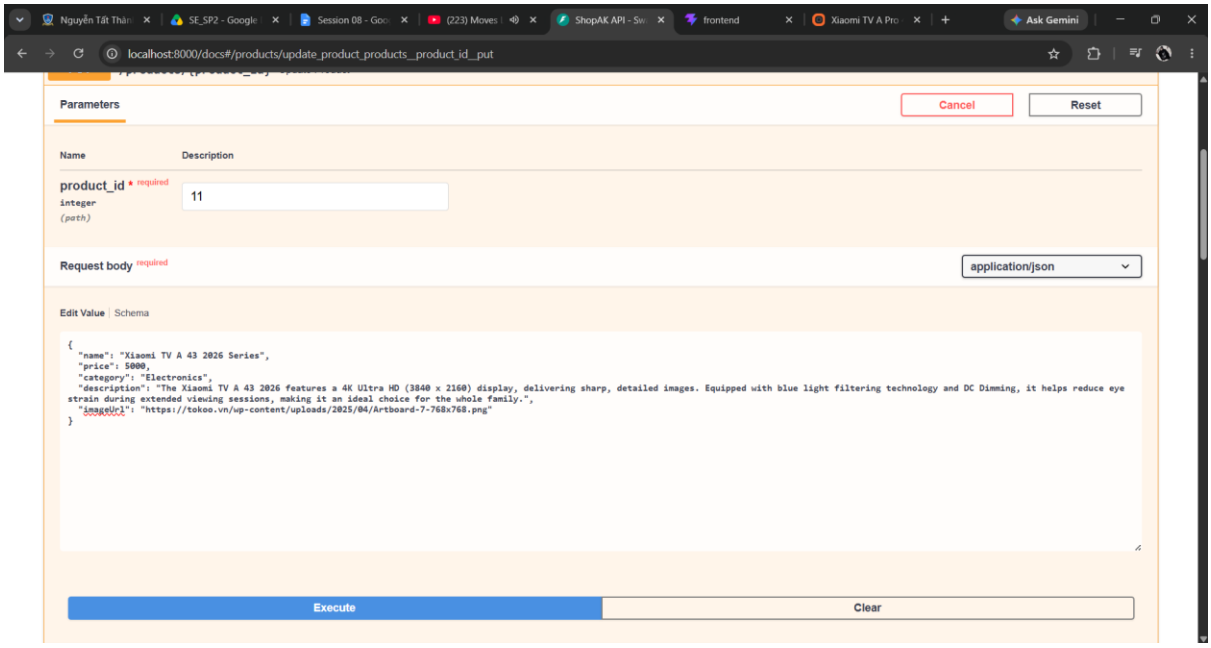


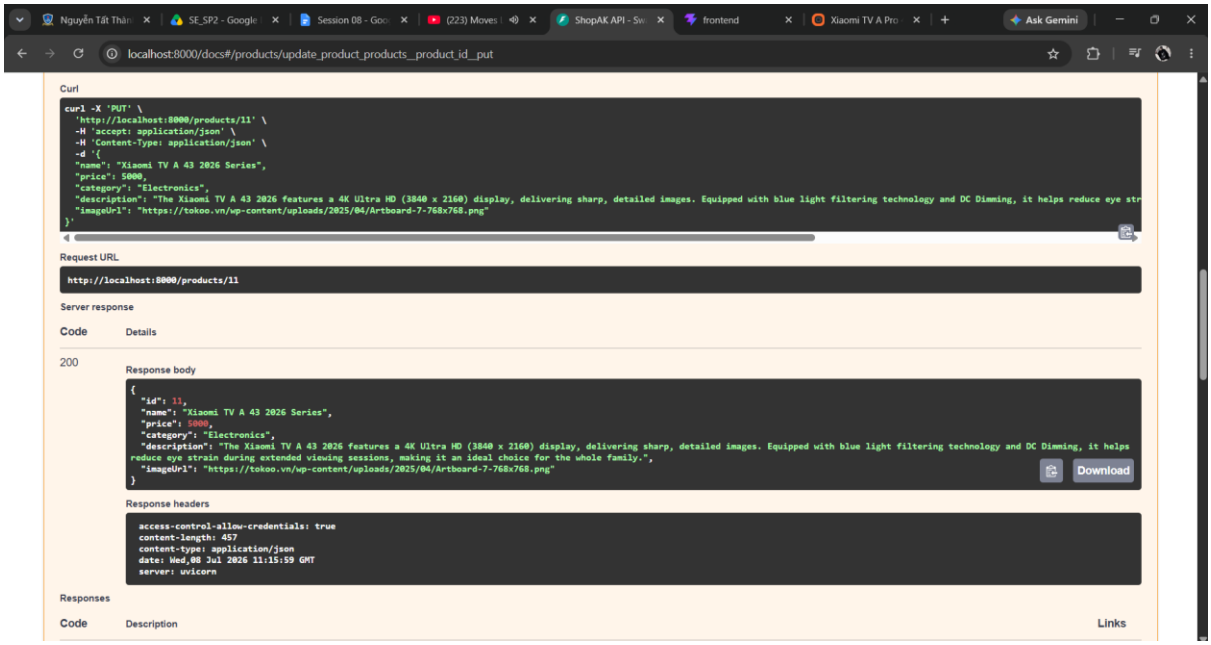
Và đây là kết quả hiển thị của web



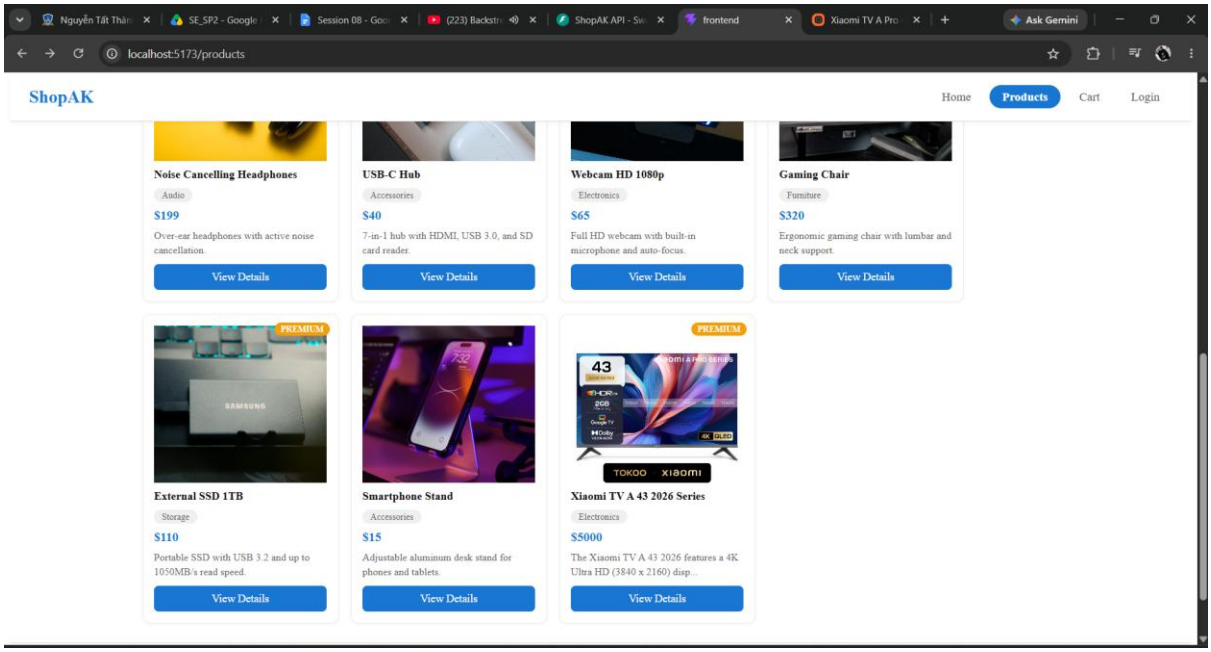


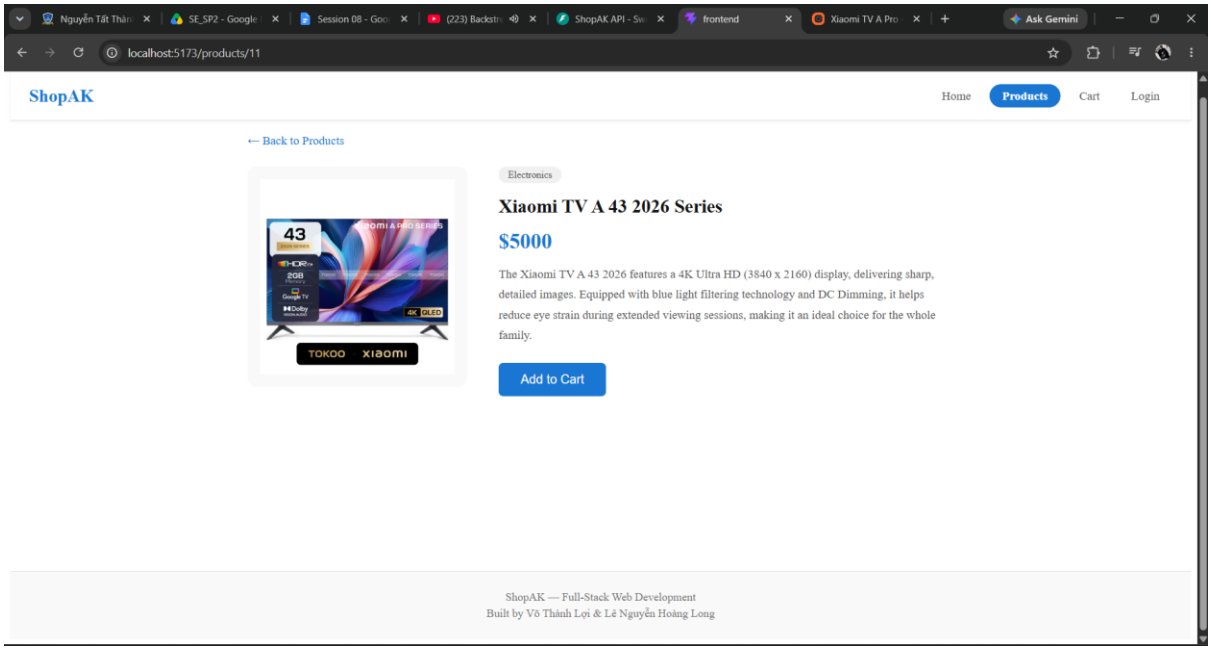
Tiếp theo chúng ta sẽ test về PUT /products/{id} update sản phẩm vừa tạo và trả về kết quả response 200



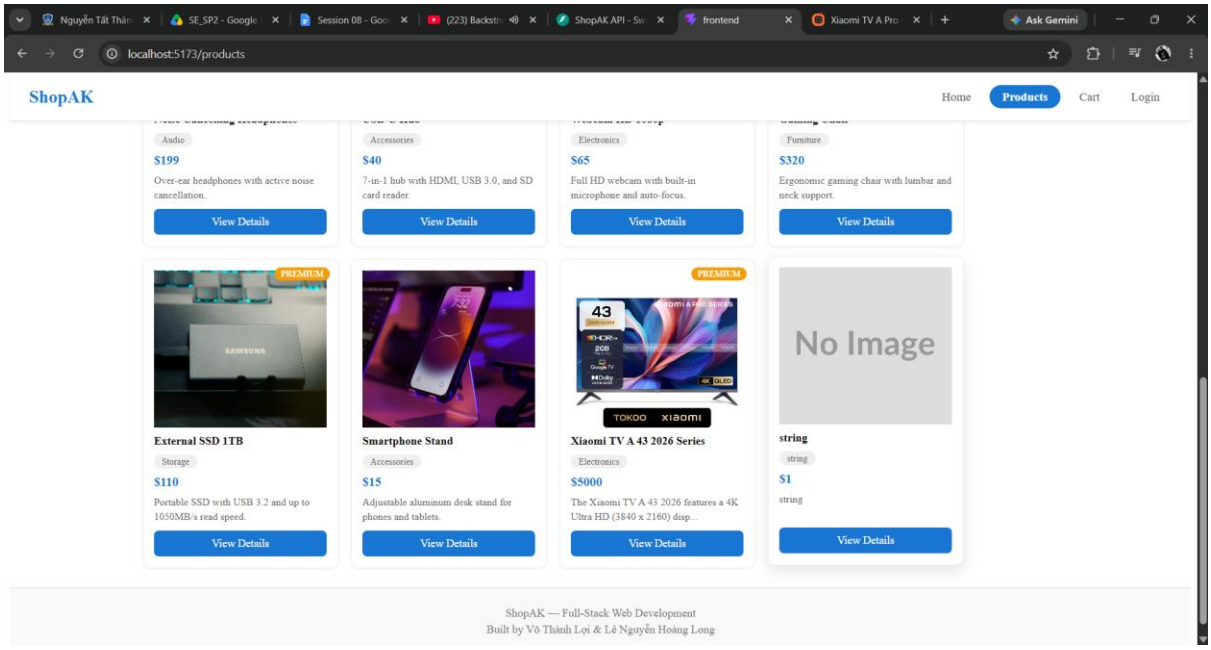


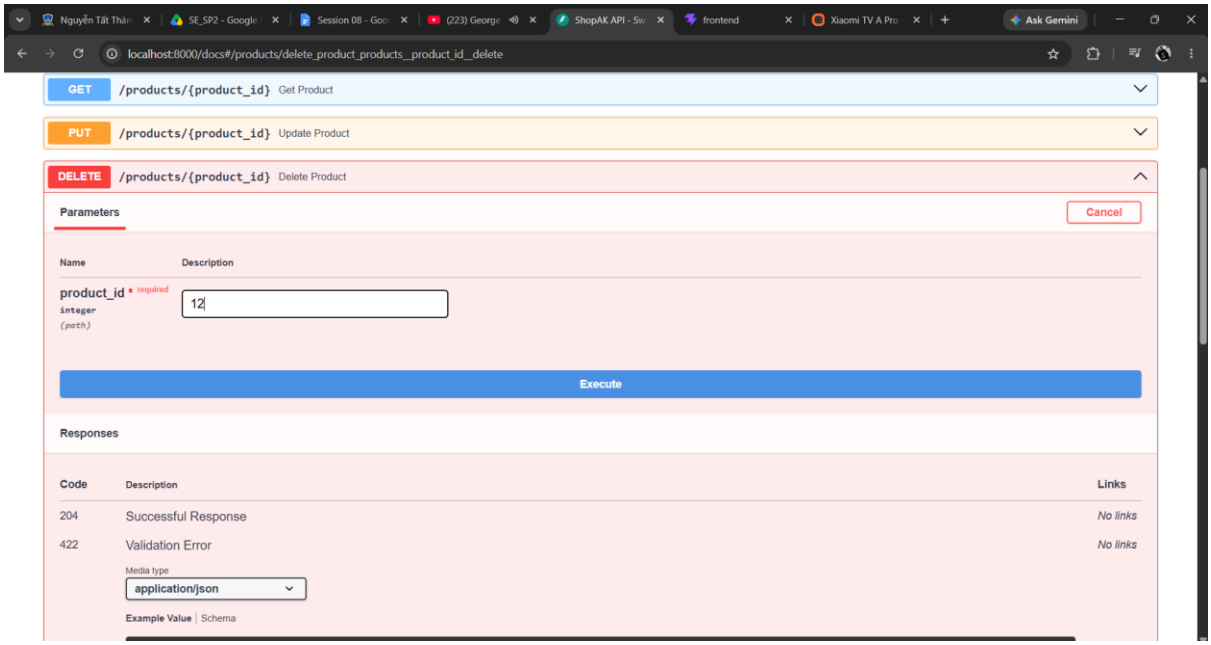
Kết quả hiển thị trên web:

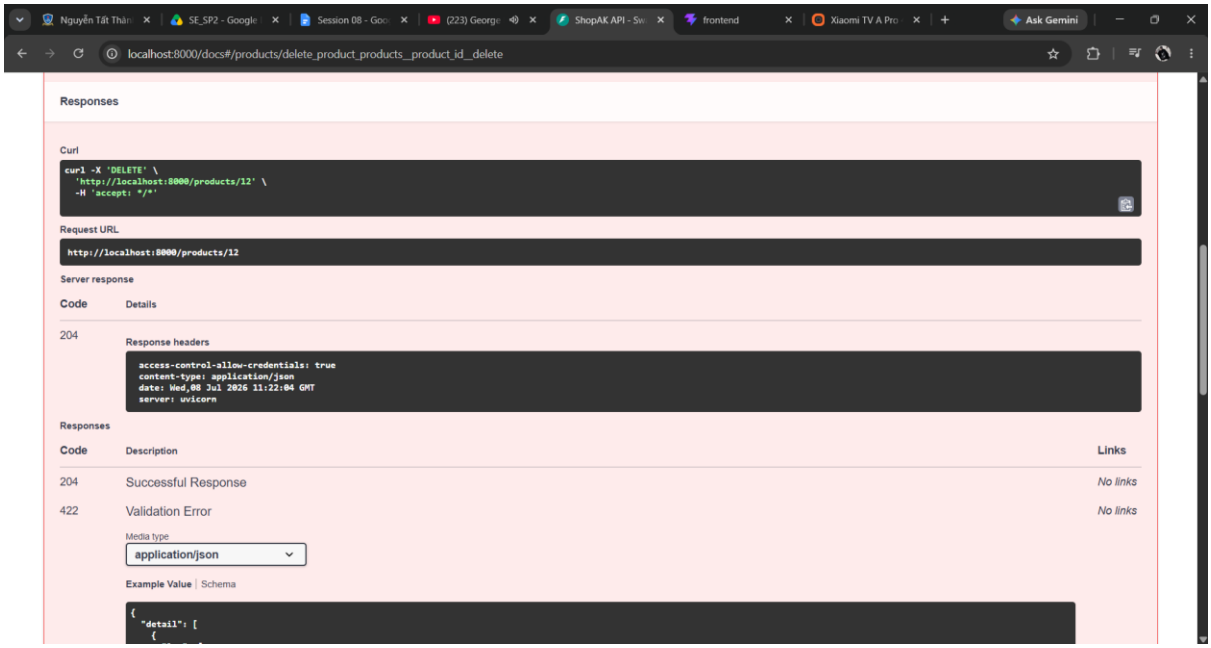




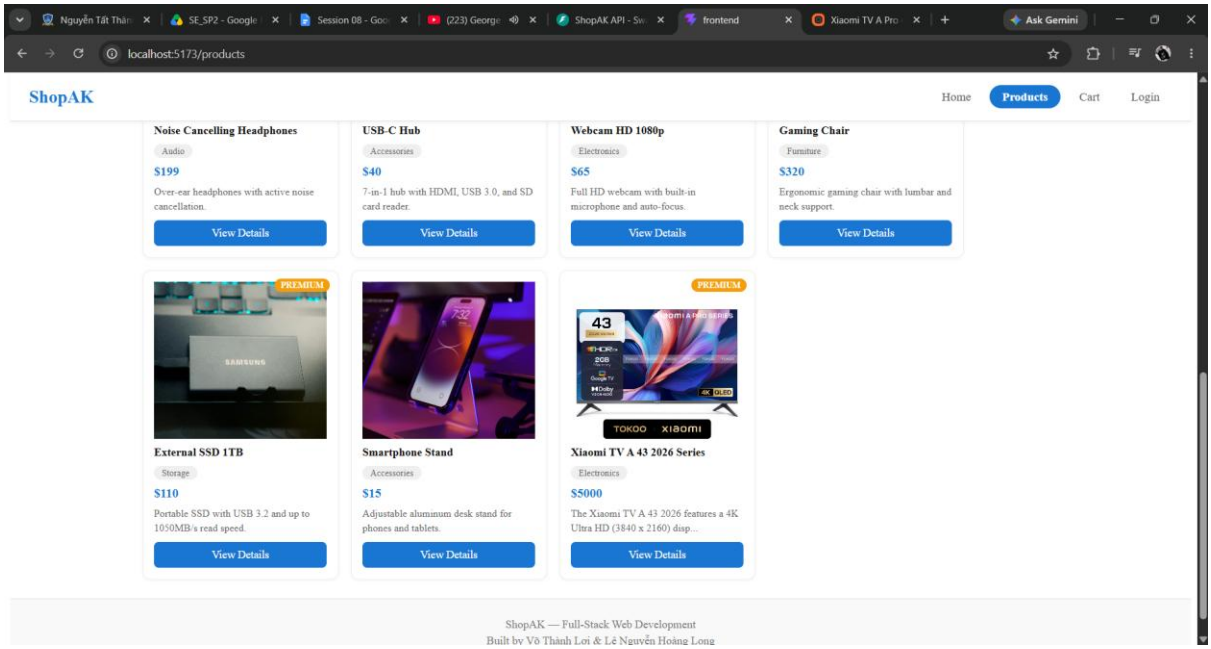
Tiếp theo chúng ta sẽ test delete sản phẩm đã tạo bằng DELETE /products/{id} sau khi xóa sẽ trả về kết quả response 204. Ở đây chúng ta sẽ tạo sản phẩm riêng để xóa


localhost:5173/products
ShopAK
Home
Products
Cart
Login
Audio
$199
Over-ear headphones with active noise cancellation.
View Details
Accessories
$40
7-in-1 hub with HDMI, USB 3.0, and SD card reader.
View Details
Electronics
$65
Full HD webcam with built-in microphone and auto-focus.
View Details
Furniture
$320
Ergonomic gaming chair with lumbar and neck support.
View Details
PREMIUM
External SSD 1TB
Storage
$110
Portable SSD with USB 3.2 and up to 1050MB/s read speed.
View Details
Smartphone Stand
Accessories
$15
Adjustable aluminum desk stand for phones and tablets.
View Details
PREMIUM
Xiaomi TV A 43 2026 Series
Electronics
$5000
The Xiaomi TV A 43 2026 features a 4K Ultra HD (3840 x 2160) disp...
View Details
No Image
string
string
$1
string
View Details
ShopAK — Full-Stack Web Development
Built by Võ Thành Lợi & Lê Nguyễn Hoàng Long

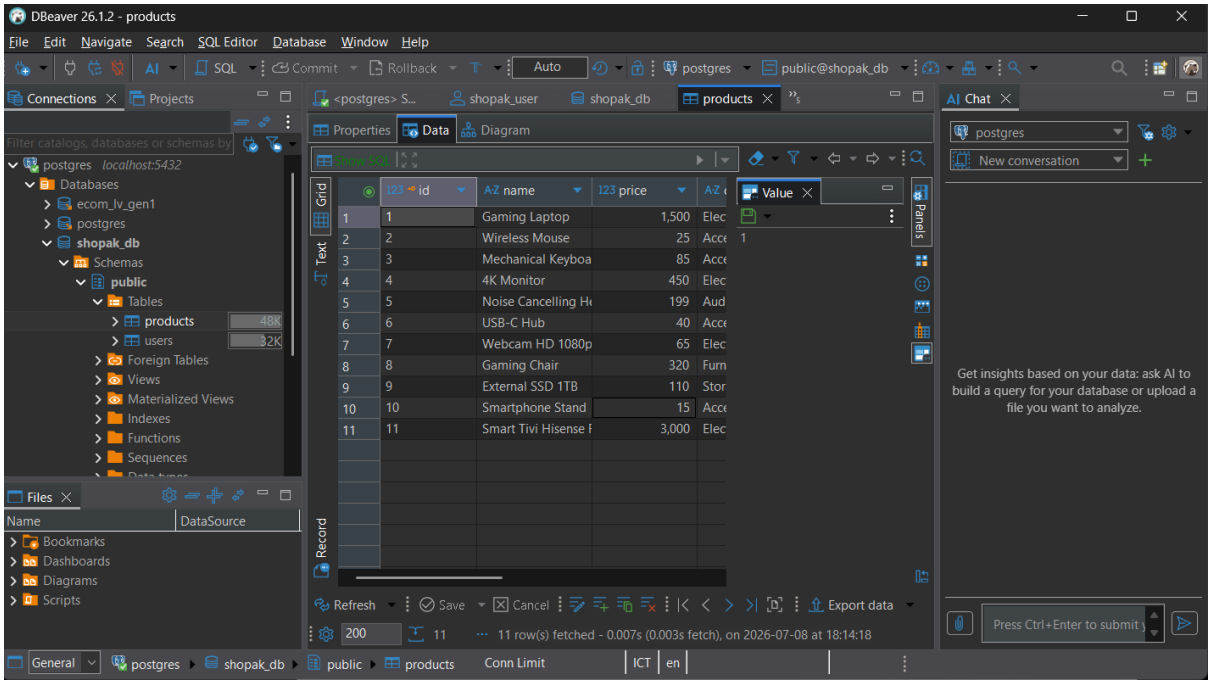

localhost:8000/docs#/products/delete_product_products__product_id__delete
GET /products/{product_id} Get Product
PUT /products/{product_id} Update Product
DELETE /products/{product_id} Delete Product
Parameters
Cancel
Name
Description
product_id * required
integer
(path)
12
Execute
Responses
Code
Description
Links
204
Successful Response
No links
422
Validation Error
No links
Media type
application/json
Example Value | Schema



Và đây là kết quả sau khi xóa trên web sẽ hiển thị không còn sản phẩm đã tạo

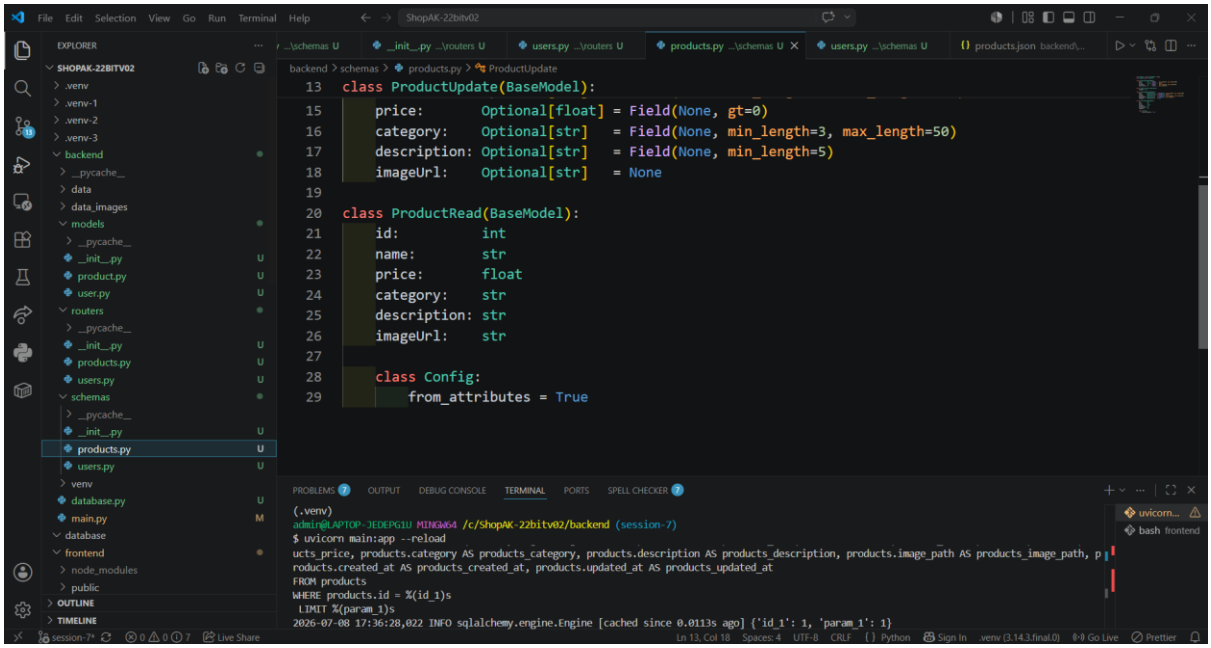


POST/PUT qua Swagger thực sự nằm trong database và đây là bằng chứng "persists"

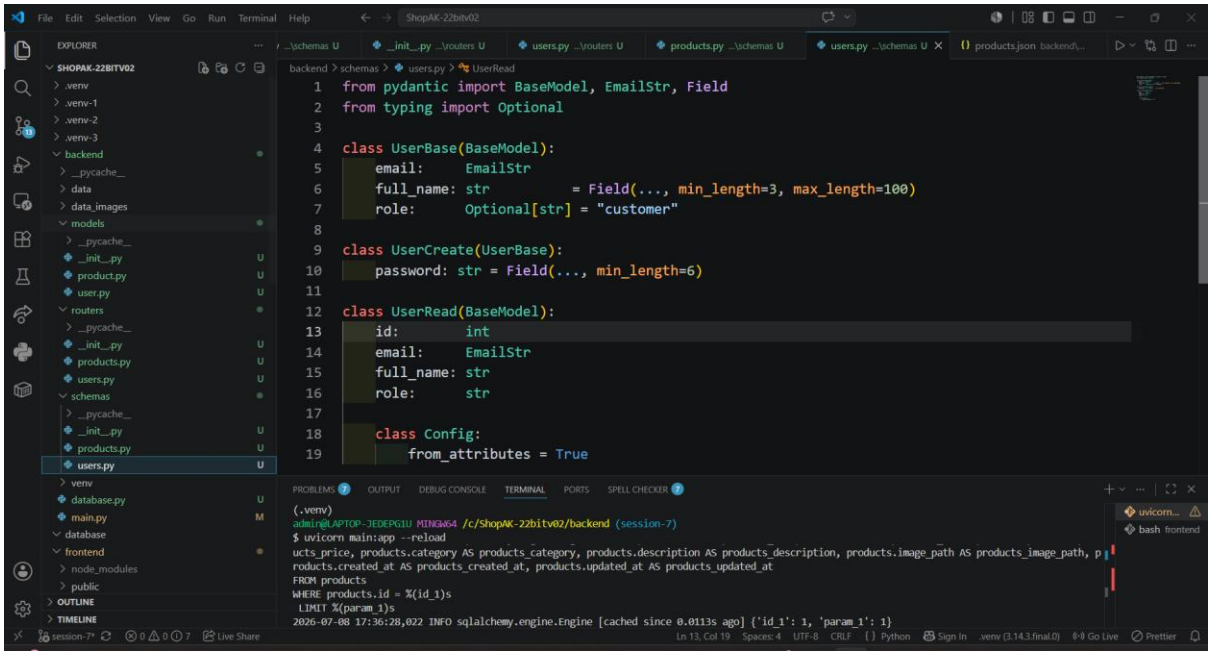


Quan sát ProductRead và UserRead ẩn field nhạy cảm

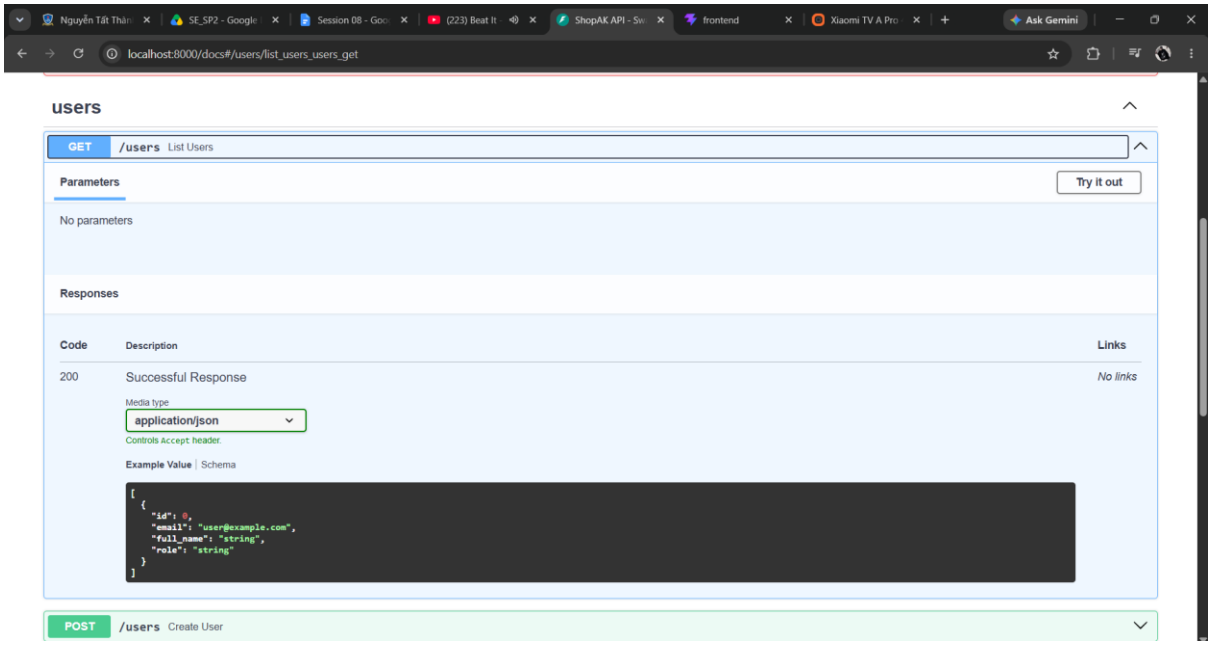
Để làm được thì chúng em đã thêm class ProductRead



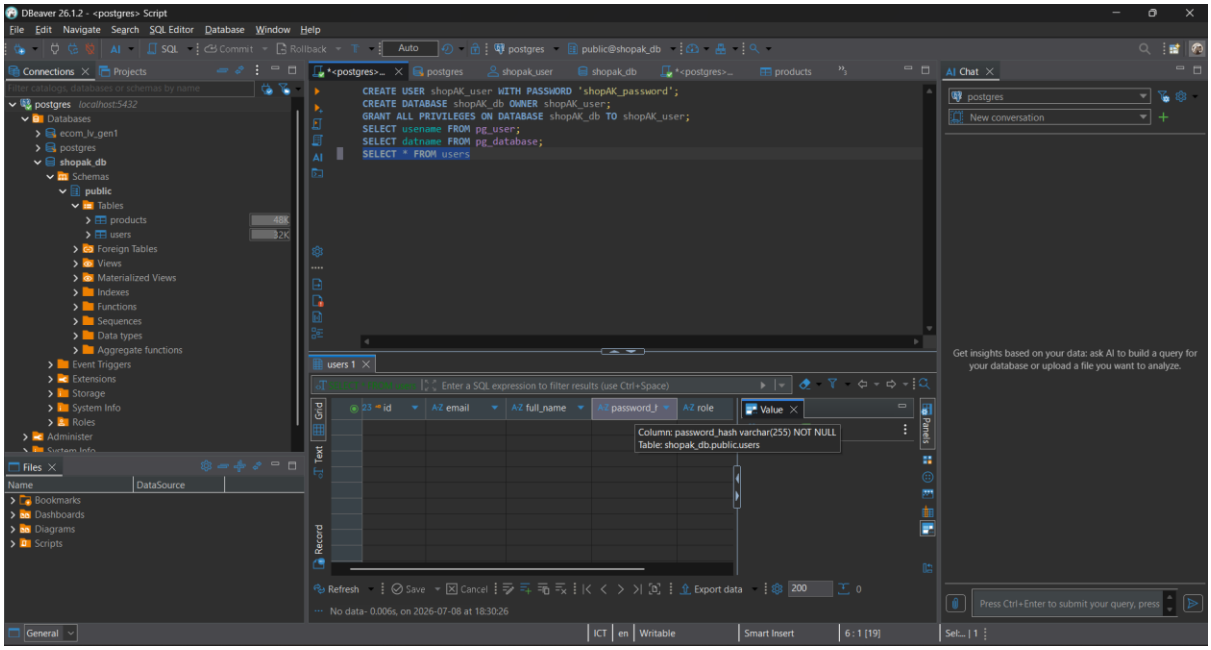
Và class UserRead



Trong Swagger khi chúng ta gọi GET /users sẽ thấy chỉ có (id, email, full_name, role, không có password_hash)



Đối chiếu với sql Dbeaver chúng ta chạy SELECT * FROM users chúng ta sẽ thấy cột password_hash **có tồn tại trong DB** nhưng không xuất hiện trong API response chứng minh schema tách biệt hoạt động đúng.



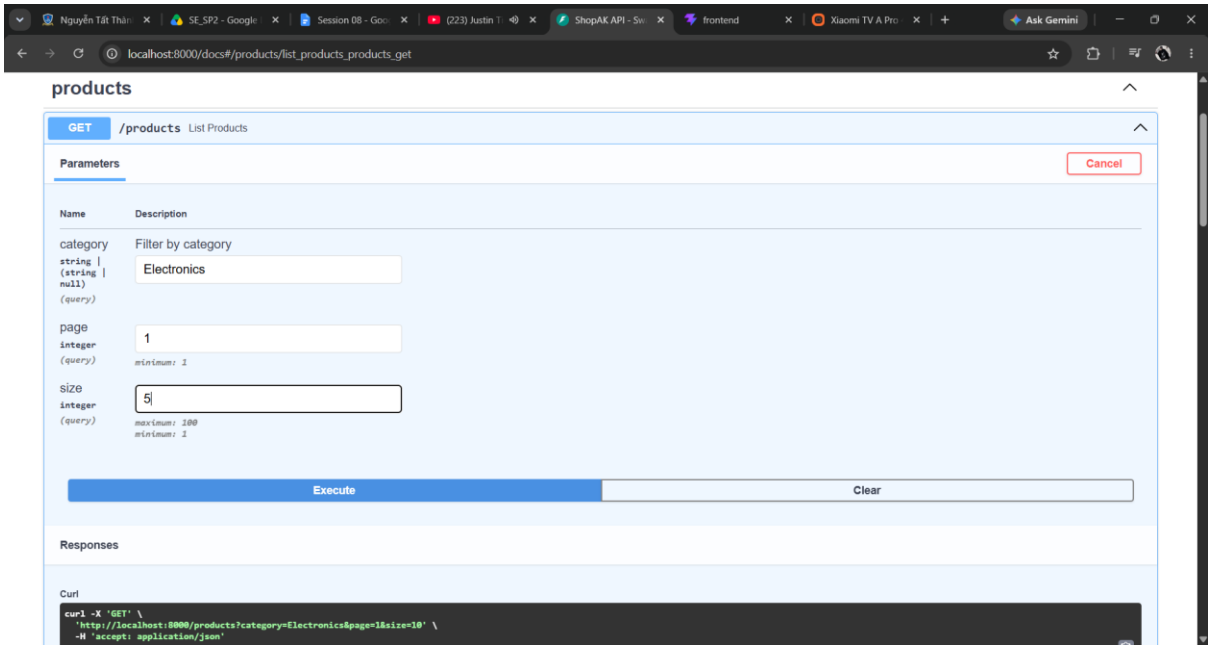
Homework

1. Filter & pagination cho GET /products

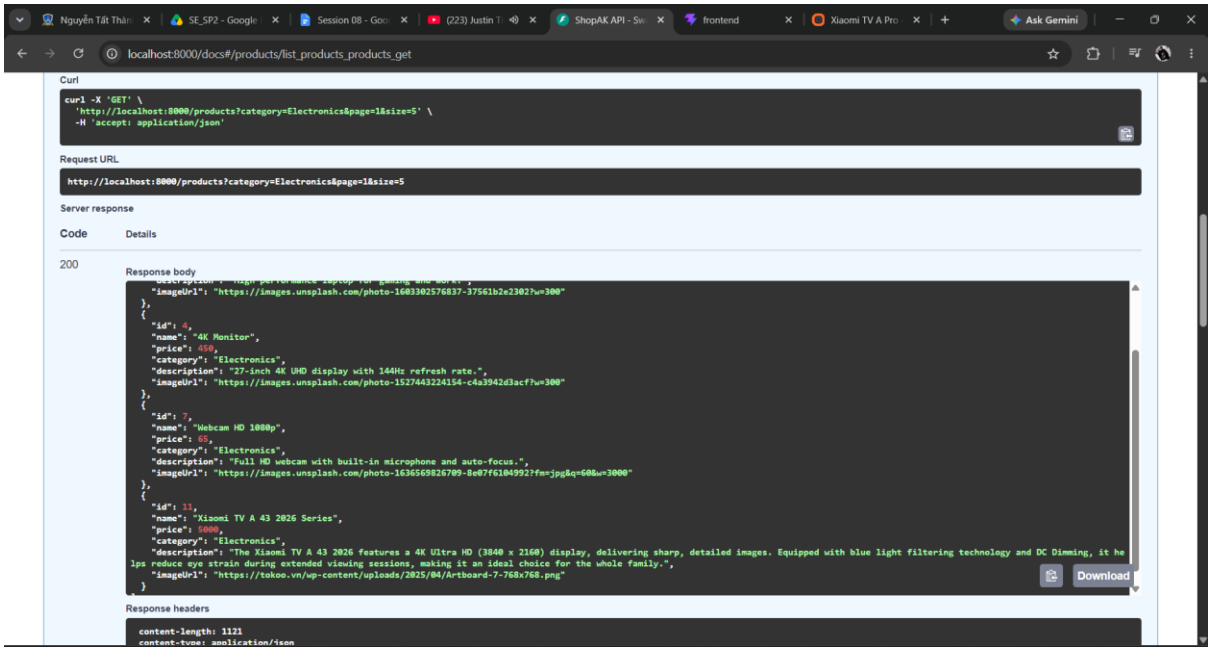
Ở đây chúng ta sẽ có: **(Query(None), .filter(), .offset().limit())**

```python
backend > routers > products.py > list_products
13  def list_products(
17      db: Session = Depends(get_db),
18  ):
19      query = db.query(ProductDB)
20
21      if category:
22          query = query.filter(ProductDB.category.ilike(category))
23
24      offset = (page - 1) * size
25      products = query.offset(offset).limit(size).all()
26
27      return [
28          ProductRead(
29              id=p.id, name=p.name, price=p.price,
30              category=p.category, description=p.description,
31              imageUrl=p.image_path,
32          )
33          for p in products
34      ]
```

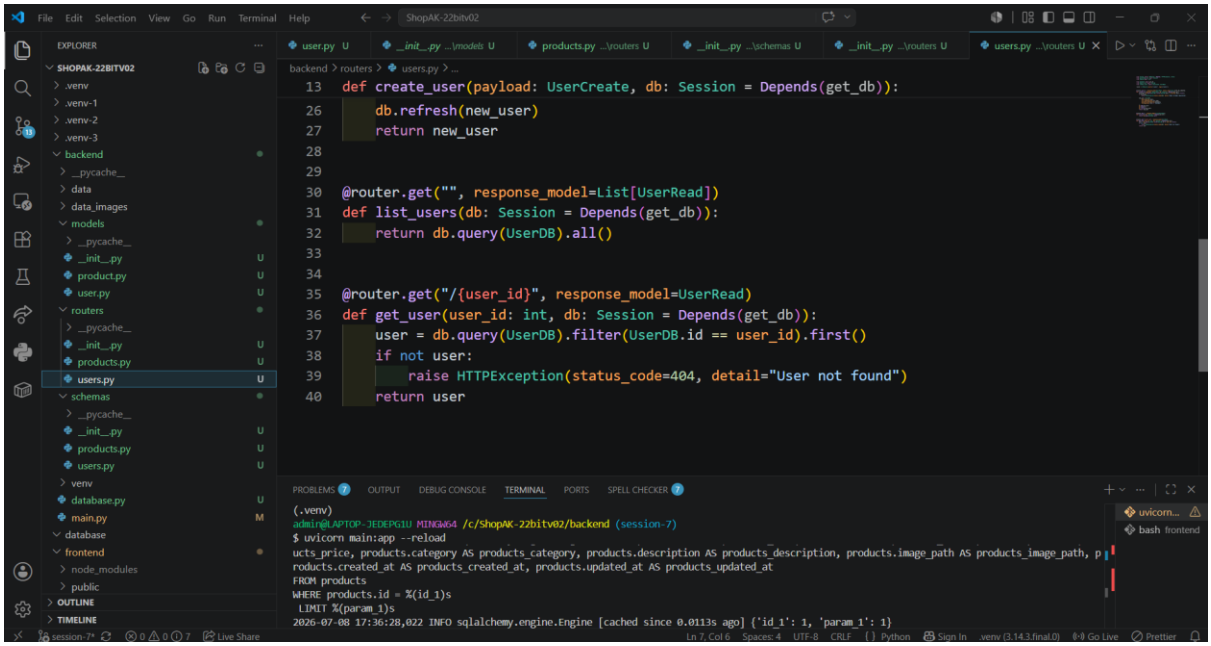
Trong Swagger chúng ta sẽ test GET /products và response chỉ trả về sản phẩm Electronics, giới hạn 5 items



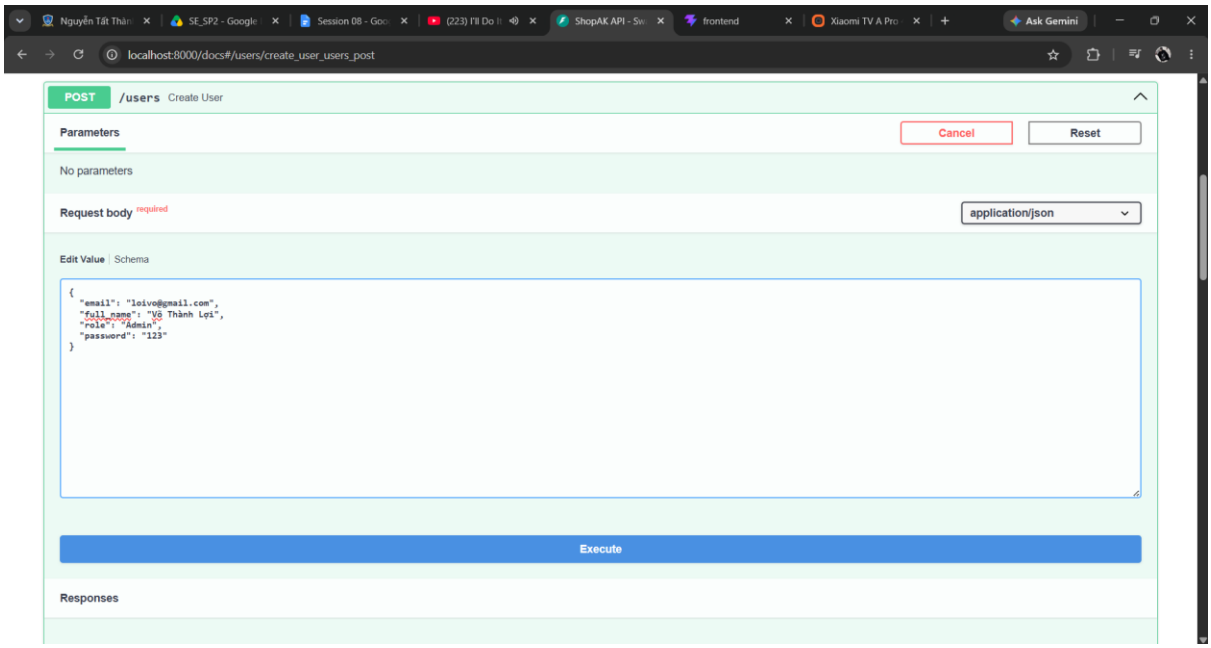
Và kết quả 200 sẽ trả về kết quả về category Electronics là dưới 5 sản phẩm



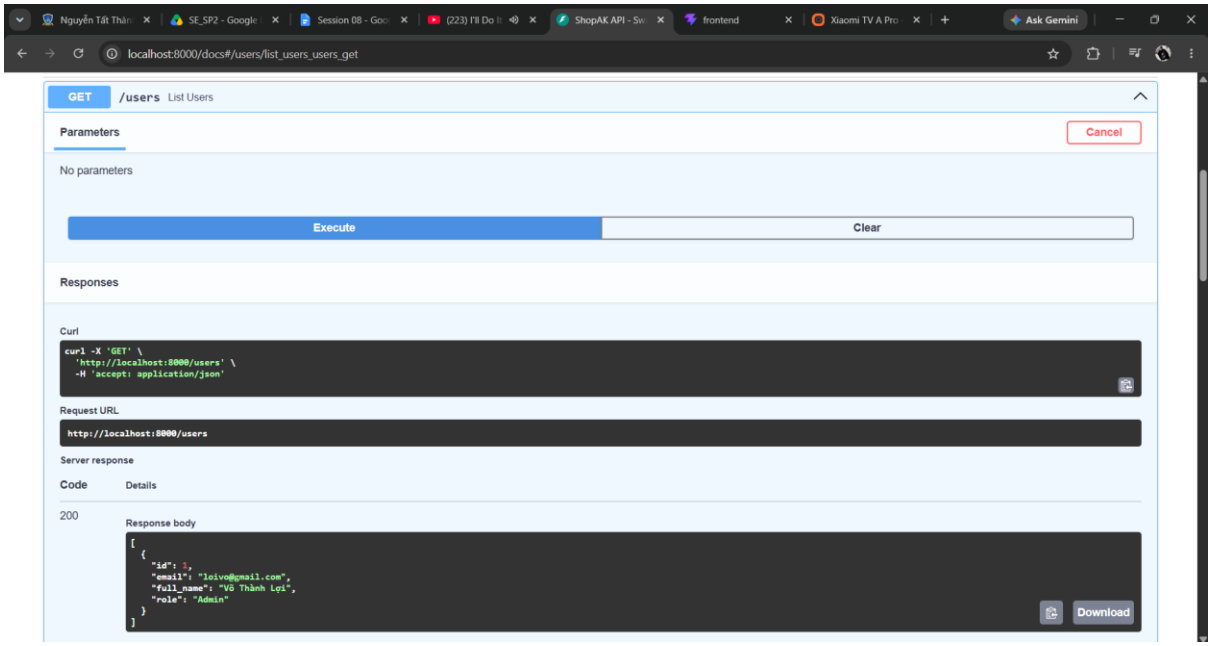
2. GET /users và GET /users/{id}



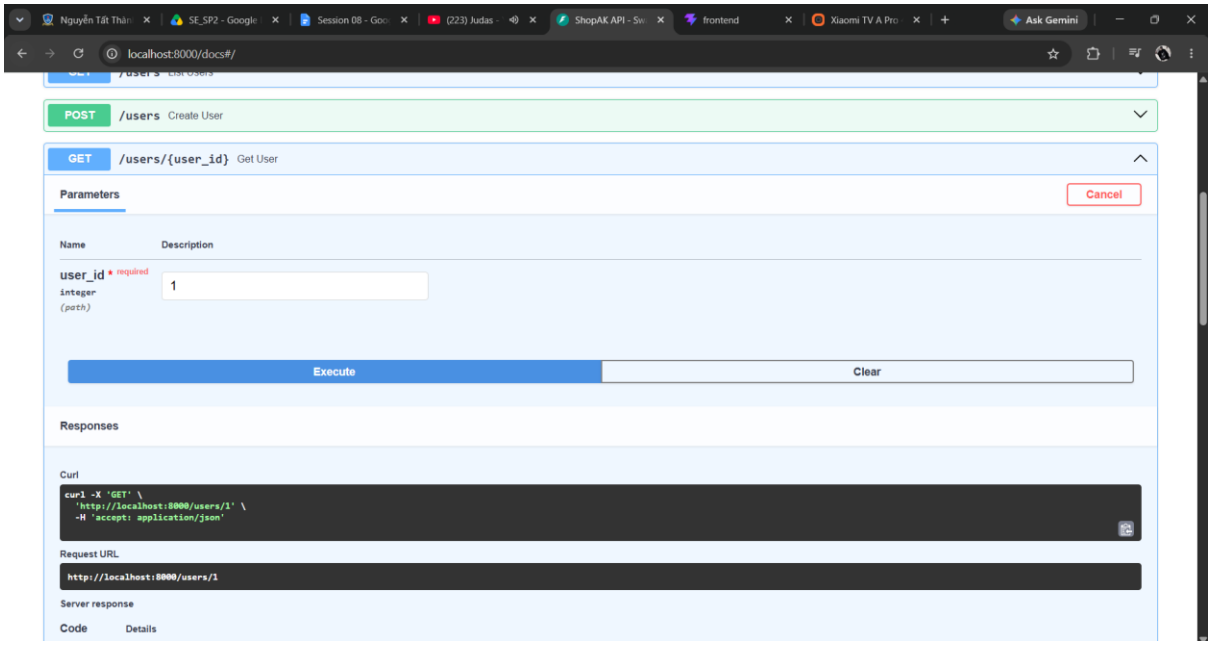
Đầu tiên chúng ta sẽ test **GET /users** trong Swagger bằng cách tạo 1 user mới hoàn toàn bằng POST/users

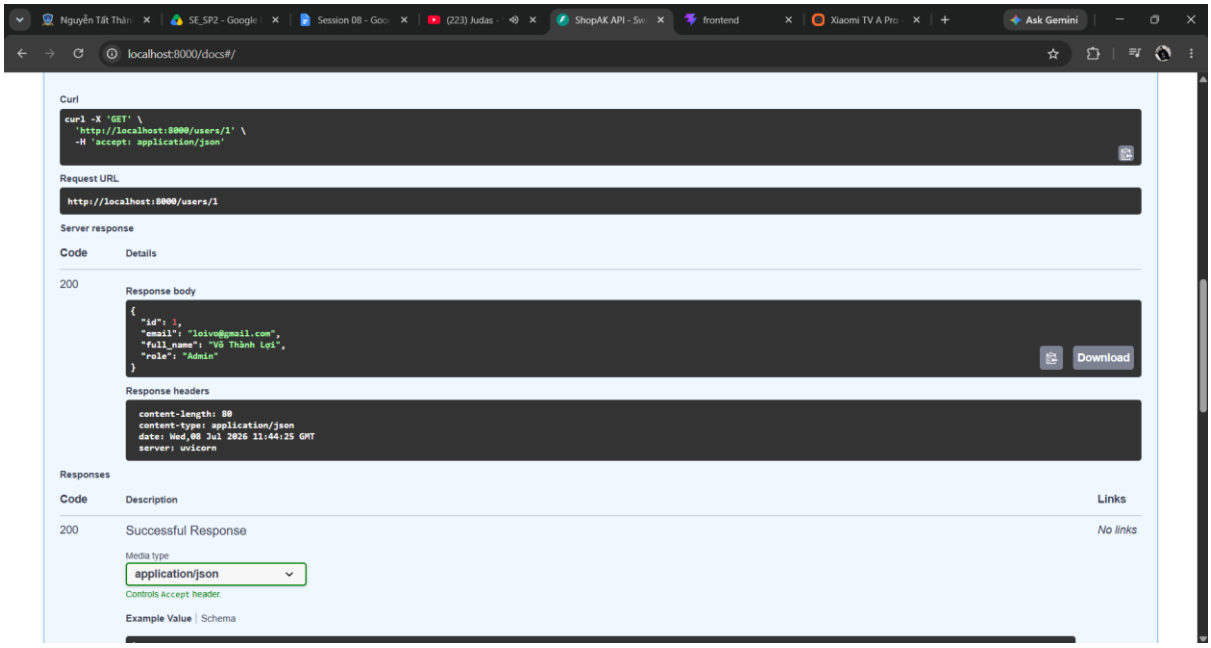


Và chúng ta sẽ GET/users bằng list users trả về kết quả gồm bao nhiêu user hiện có



1 cách chi tiết hơn bằng cách dùng **GET /users/{id}** với 1 ID cụ thể





Và mình có thể điền ID tùy theo mình chọn.

Why using PostgreSQL is better than JSON files for multi-user apps.

JSON file không hỗ trợ truy cập đồng thời khi nhiều người dùng cùng ghi dữ liệu, có thể xảy ra race condition làm mất hoặc hỏng dữ liệu. PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ cấp production, hỗ trợ transaction, indexing, truy vấn phức tạp và đảm bảo tính toàn vẹn dữ liệu (ACID) ngay cả khi có nhiều request đồng thời.

How separating DB models and Pydantic schemas improves security and maintainability.

DB model (ProductDB, UserDB) đại diện cho cấu trúc bảng thực tế, có thể chứa trường nhạy cảm như password_hash. Pydantic schema (ProductRead, UserRead) chỉ định nghĩa những gì client được phép thấy, giúp ẩn hoàn toàn dữ liệu nhạy cảm khỏi response. Việc tách biệt này cũng cho phép thay đổi cấu trúc database mà không ảnh hưởng trực tiếp API, giúp hệ thống dễ bảo trì và mở rộng hơn.